

PROJETO EM GRUPO (2,0)

TEMA: Desenvolvimento de API em Spring Boot de acordo com as premissas e práticas da programação orientada a objetos

RECURSOS – ferramentas, frameworks: IntelliJ Ultimate, SPRING BOOT (framework), JDK, JVM, Maven, Docker.

ROTEIRO PARA REALIZAÇÃO DO PROJETO

- 1) Alunos divididos em equipes
- 2) Leitura do Estudo de Caso e Desafio:

Estudo de Caso (Desafio): Implementando User Story com TDD e BDD, considerando o método ATDD.

GAMIFICAÇÃO PARA ENGAJAMENTO DE EDUCAÇÃO CONTINUADA

Uma determinada plataforma vende cursos online e EAD no modelo de assinaturas. O aluno paga um valor mensal e tem acesso a um conjunto de cursos para assinatura básica. A cada curso terminado e com média acima de 7,0, o aluno tem direito a realização de mais 3 cursos. O aluno que escrever mais tópicos no fórum e ajudar outros participantes com seus comentários, ganha um curso no final do mês. Quando o aluno conquistar 12 cursos, seu plano de assinatura passa a ser “Premium” e ele passa a receber voucher para participar de projetos reais, durante os cursos, e receber 3 moedas, que podem ser convertidas em conhecimento (novos cursos), acumular ou receber por criptomoeda.

Responda:

- 3) A partir do estudo de caso-desafio (supramencionado), defina uma funcionalidade. A partir disso, faça:
Plataforma EAD com progressão modular, gamificação e upgrade automático de plano. Alunos se matriculam em cursos, concluem módulos sequencialmente, ganham moedas e cursos extras como recompensa, e fazem upgrade para Premium após 12 cursos concluídos.
- 4) Defina os objetos.

Objeto	Descrição
Aluno	Estudante com progresso, saldo de moedas, plano
Professor	Instrutor com especialidade e remuneracao
Curso	Agregado com modulos e aulas
Modulo	Unidade sequencial dentro de um curso
Aula	Conteudo (video/PDF/link) dentro de um modulo
Matricula	Vínculo aluno-curso com progresso
ProjetoFinal	Entrega avaliada do aluno
TransacaoMoeda	Historico de movimentacao de moedas
Assinatura	Plano de assinatura do aluno

VO	Descrição
Email	Validacao de formato de email
NomeCompleto	Nome nao vazio
SenhaCriptografada	Hash BCrypt da senha
CarteiraDigital	Endereco e rede blockchain
NivelCurso	Dificuldade do curso

5) Defina as classes correspondentes

Pessoa.java (classe abstrata base) - src/main/java/com/emplo/usuariosimples/domain/usuario/entity/Pessoa.java:13-56

```
@Entity
@Table(name = "pessoa")
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Pessoa {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
    @Embedded private NomeCompleto nome;
    @Embedded private Email email;
    @Embedded private SenhaCriptografada senha;
    private boolean ativo;
    private LocalDate dataNascimento;
    private LocalDateTime dataCadastro;
    @Enumerated(EnumType.STRING) private PerfilUsuario perfil;
    // ...
}
```

Aluno.java (com metodos de negocio) - src/main/java/com/emplo/usuariosimples/domain/usuario/entity/Aluno.java:14-75

```
@Entity @Table(name = "aluno")
@PrimaryKeyJoinColumn(name = "pessoa_id")
public class Aluno extends Pessoa {
    private int totalCursosConcluidos;
    private int saldoCursosExtras;
    private int saldoMoedas;
    @Enumerated(EnumType.STRING) private TipoPlano tipoPlano;

    public void receberCursosExtras(int quantidade) { this.saldoCursosExtras += quantidade; }
    public void fazerUpgradePremium() { this.tipoPlano = TipoPlano.PREMIUM; }
    public void creditarMoedas(int quantidade) { this.saldoMoedas += quantidade; }
    public boolean isPremium() { return TipoPlano.PREMIUM.equals(this.tipoPlano); }
    public void incrementarCursosConcluidos() { this.totalCursosConcluidos++; }
}
```

6) Defina o relacionamento entre as classes.

```
Pessoa (abstrata, JOINED)
├─ Aluno      (@PrimaryKeyJoinColumn)
├─ Professor  (@PrimaryKeyJoinColumn)
└─ Administrador (@PrimaryKeyJoinColumn)

Curso 1 — * Modulo (@OneToMany EAGER, cascade ALL)
Modulo 1 — * Aula (@OneToMany EAGER, cascade ALL)
Modulo * — 1 Curso (@ManyToOne LAZY)

Matricula — referencia Aluno por UUID (alunoId)
Matricula — referencia Curso por Long (cursoId)

Aluno 1 — * TransacaoMoeda (por alunoId UUID)

ProjetoFinal — referencia Matricula (matriculaId)
ProjetoFinal — referencia Aula (aulaId)
```

```

v main
  v java
    v com.emplo.usuariosimples
      > application
      v domain
        v academico
          > entity
          > enums
          > evento
          > repository
          > service
          > valueobject
        > engajamento
        > financeiro
        > gamificacao
        > usuario

```

7) Crie o projeto no Spring Boot, utilizando o Maven para gestão de build e dependência. Utilize o IntelliJ Ultimate. Inclua as dependências: Spring WEB, Spring Data, Banco de Dados em Memória H2 e Banco de Dados SQLPostGres.

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.3.5</version>
</parent>
<dependencies>
  <dependency> <!-- Spring WEB -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency> <!-- Spring Data JPA -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency> <!-- H2 Database (dev) -->
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency> <!-- PostgreSQL (prod) -->
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency> <!-- Swagger/OpenAPI -->
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.5.0</version>
  </dependency>
</dependencies>
```

- 8) No projeto e de acordo com os princípios de Clean Architecture, crie as camadas: Entity e VO, Service, Repository<. Aqui orientação a objetos é o foco principais e os padrões relacionados.

Value Objects:

Email.java - src/main/java/com/emplo/usuariosimples/domain/usuario/valueobject/Email.java:1-16

```
@Embeddable
public record Email(
    @Column(name = "email", nullable = false, unique = true, length = 120)
    String endereco
) {
    public Email {
        if (endereco == null || !endereco.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$"))
            throw new IllegalArgumentException("E-mail invalido");
    }
}
```

Repository (interface de dominio):

CursoRepository.java - src/main/java/com/emplo/usuariosimples/domain/academico/repository/CursoRepository.java:1-20

```
public interface CursoRepository {
    Optional<Curso> findById(Long id);
    List<Curso> findByStatus(StatusCurso status);
    List<Curso> findAll();
    Curso save(Curso curso);
    void deleteById(Long id);
}
```

Domain Services:

ProgressaoModularService.java -

src/main/java/com/emplo/usuariosimples/domain/academico/service/ProgressaoModularService.java:1-26

```
@Service
public class ProgressaoModularService {
    public boolean podeAvancarModulo(Matricula matricula, int ordemModuloAlvo) {
        if (!matricula.podeAvancarModulo()) return false;
        int proximoModulo = matricula.getModulosConcluidos() + 1;
        return ordemModuloAlvo == proximoModulo;
    }
    // ...
}
```

9) Crie as camadas DTO (Data Transfer Object) e Controller (Padrão Restful).

DTOs:

LoginRequestDTO.java - src/main/java/com/emplo/usuariosimples/interfaces/rest/usuario/dto/LoginRequestDTO.java:1-14

```
public record LoginRequestDTO(
    @NotBlank @Email String email,
    @NotBlank String senha
) {}
```

CursoResponseDTO.java (com DTOs aninhados) -

src/main/java/com/emplo/usuariosimples/interfaces/rest/academico/dto/CursoResponseDTO.java:9-82

```
public record CursoResponseDTO(
    Long id, String titulo, String descricao, String publicoAlvo,
    String conhecimentosPrevios, String nivel, String status,
    List<ModuloResponseDTO> modulos
) {
    public static CursoResponseDTO from(Curso c) { /* mapeamento */ }

    public record ModuloResponseDTO(Long id, String titulo, String descricao,
        int ordem, int totalAulas, List<AulaResponseDTO> aulas) { /* ... */ }

    public record AulaResponseDTO(Long id, String titulo, String descricao,
        String tipo, String url, String duracao, int ordem) { /* ... */ }
}
```

Controllers:

AuthController.java (register + login) - src/main/java/com/emplo/usuariosimples/interfaces/rest/usuario/AuthController.java:14-50

```
@RestController @RequestMapping("/auth")
public class AuthController {
    @PostMapping("/register")
    public ResponseEntity<Void> register(@Valid @RequestBody RegisterRequestDTO request) { ... }

    @PostMapping("/login")
    public ResponseEntity<LoginResponseDTO> login(@Valid @RequestBody LoginRequestDTO request) { ... }
}
```

CursoController.java (CRUD completo) -

src/main/java/com/emplo/usuariosimples/interfaces/rest/academico/CursoController.java:14-68

```
@RestController @RequestMapping("/cursos") @CrossOrigin
public class CursoController {
    @PostMapping // CREATE
    @GetMapping // READ (listar publicados)
    @GetMapping("/{id}") // READ (por id)
    @PutMapping("/{id}") // UPDATE
    @DeleteMapping("/{id}") // DELETE
}
```

- 10) Inserir as Configurações no projeto para apresentação da API (Application Programming Interface) via Swagger.

```
Configuracao no application.properties:

springdoc.swagger-ui.path=/swagger-ui.html

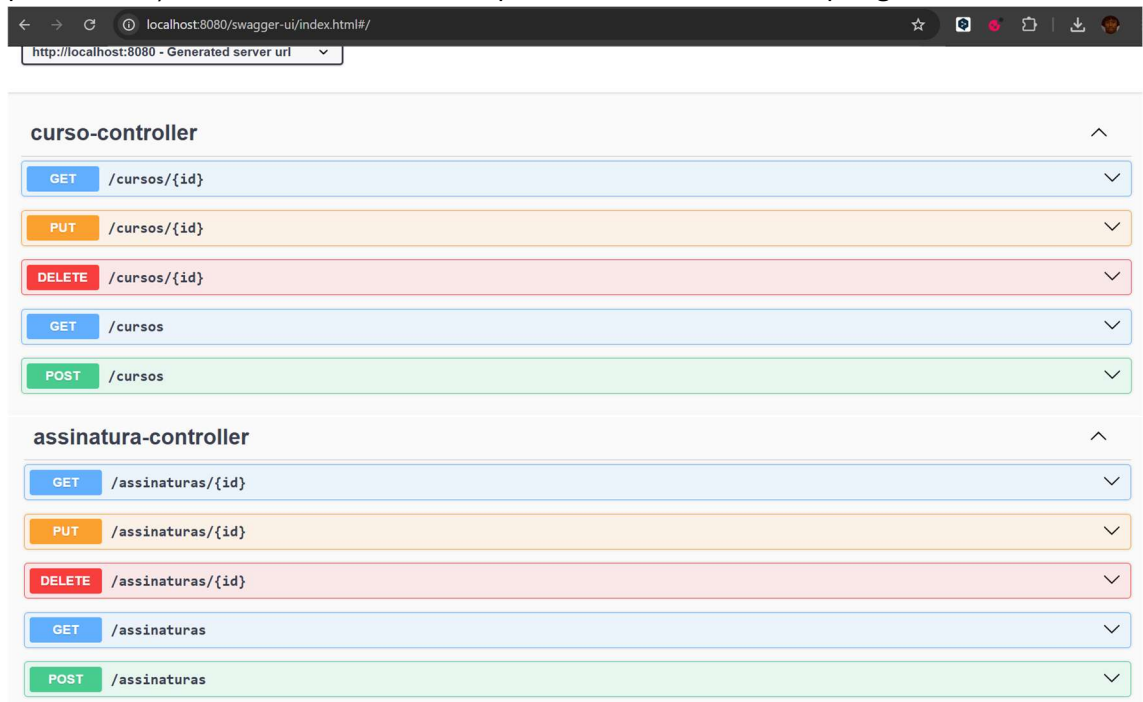
Dependencia no pom.xml:

<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.5.0</version>
</dependency>

Whitelist no SecurityConfig.java - SecurityConfig.java:40

.requestMatchers("/swagger-ui.html", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
```

- 11) Cada integrante da equipe DEVERÁ desenvolver no mínimo endpoints (get, post, put e delete), via camada controller e apresentados em URI, via Spring Boot.



projeto-final-controller		^
POST	/projetos	▼
PATCH	/projetos/{id}/avaliar	▼
GET	/projetos/{id}	▼
matricula-controller		^
GET	/matriculas	▼
POST	/matriculas	▼
POST	/matriculas/{id}/modulos/{ordem}/concluir	▼
GET	/matriculas/{id}	▼
GET	/matriculas/{id}/progresso	▼

- 12) A equipe deverá apresentar a aplicação rodando via Spring Boot, mostrando as funcionalidades implementadas na API.

```

C:\Users\ihana.nunes\projeto-spring-data-usuario-simples>mvn spring-boot:run
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.exemplo:usuariosimples >-----
[INFO] Building usuariosimples 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] >>> spring-boot:3.3.5:run (default-cli) > test-compile @ usuariosimples >>>
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ usuariosimples ---
[INFO] Copying 3 resources from src\main\resources to target\classes
[INFO] Copying 63 resources from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ usuariosimples ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 82 source files with javac [debug parameters release 17] to target\classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ usuariosimples ---
[INFO] skip non existing resourceDirectory C:\Users\ihana.nunes\projeto-spring-data-usuario-simples\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ usuariosimples ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug parameters release 17] to target\test-classes
[INFO]
[INFO] <<< spring-boot:3.3.5:run (default-cli) < test-compile @ usuariosimples <<<
[INFO]
[INFO] --- spring-boot:3.3.5:run (default-cli) @ usuariosimples ---

```

localhost:8080/swagger-ui/index.html/

Swagger **v0** **OAS 3.0** /v3/api-docs **Explore**

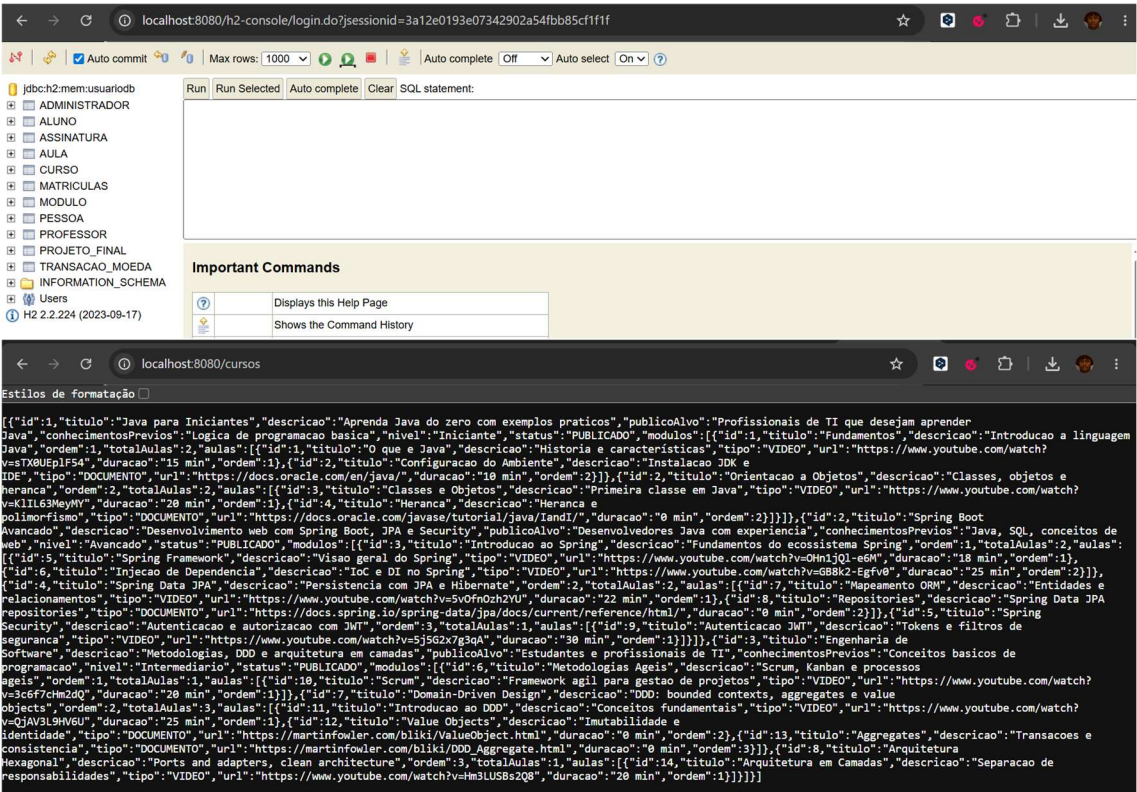
OpenAPI definition **v0** **OAS 3.0** /v3/api-docs

Servers

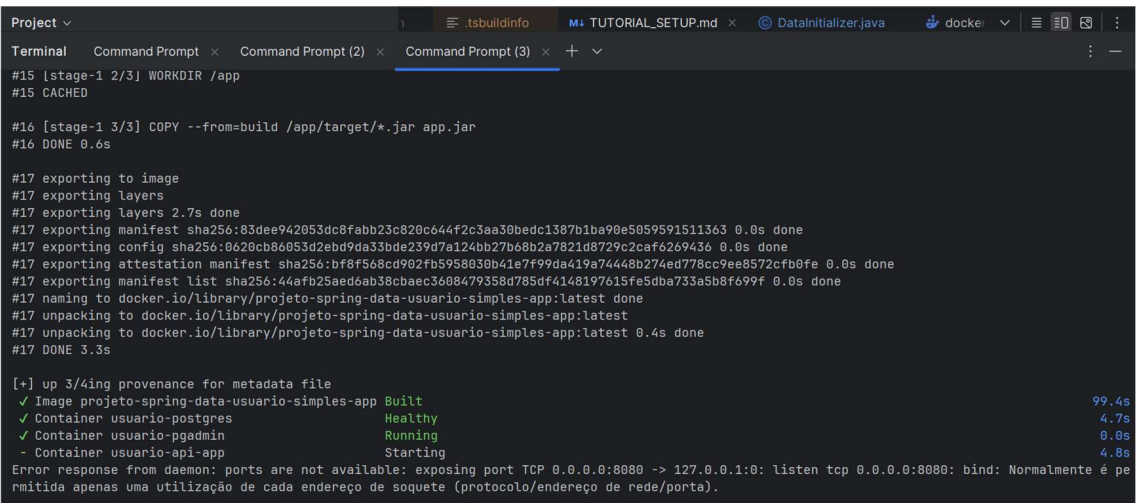
http://localhost:8080 - Generated server url

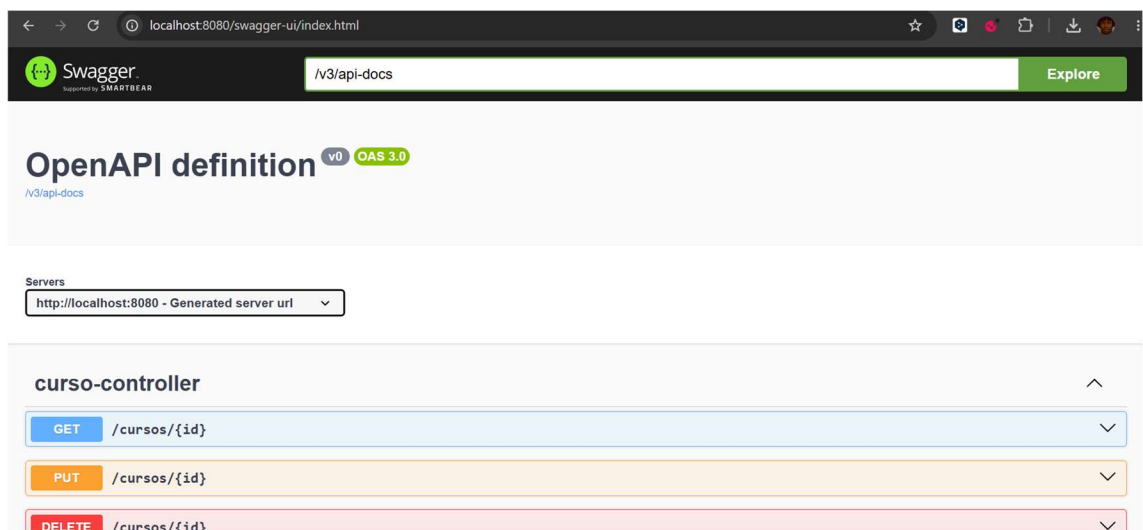
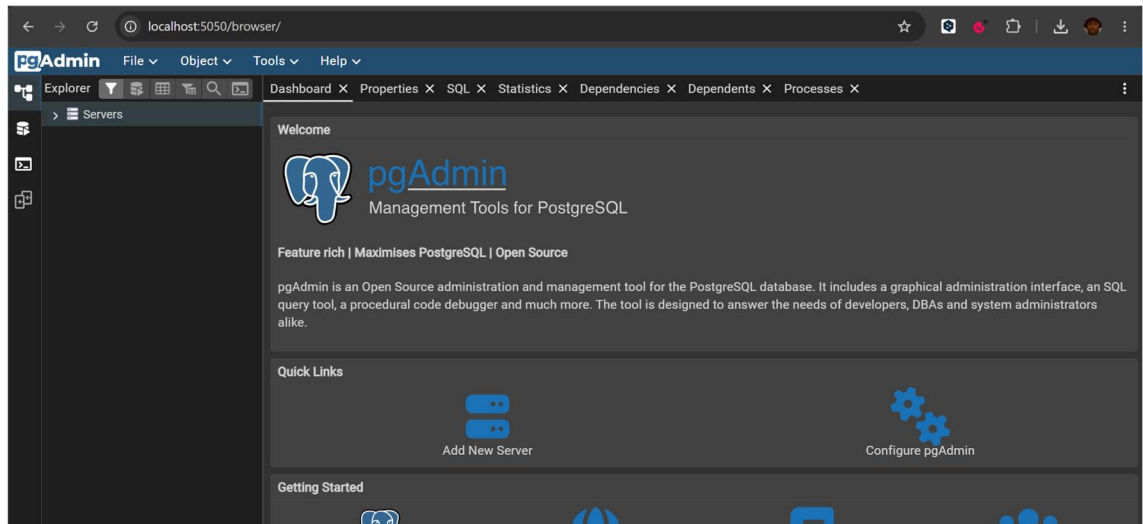
curso-controller	▼
assinatura-controller	▼
projeto-final-controller	▼
matricula-controller	▼

POO – Programação Orientada a Objetos – Profa Andréia D. de Leles



13) A equipe deverá apresentar a aplicação (API) funcionando via Docker.

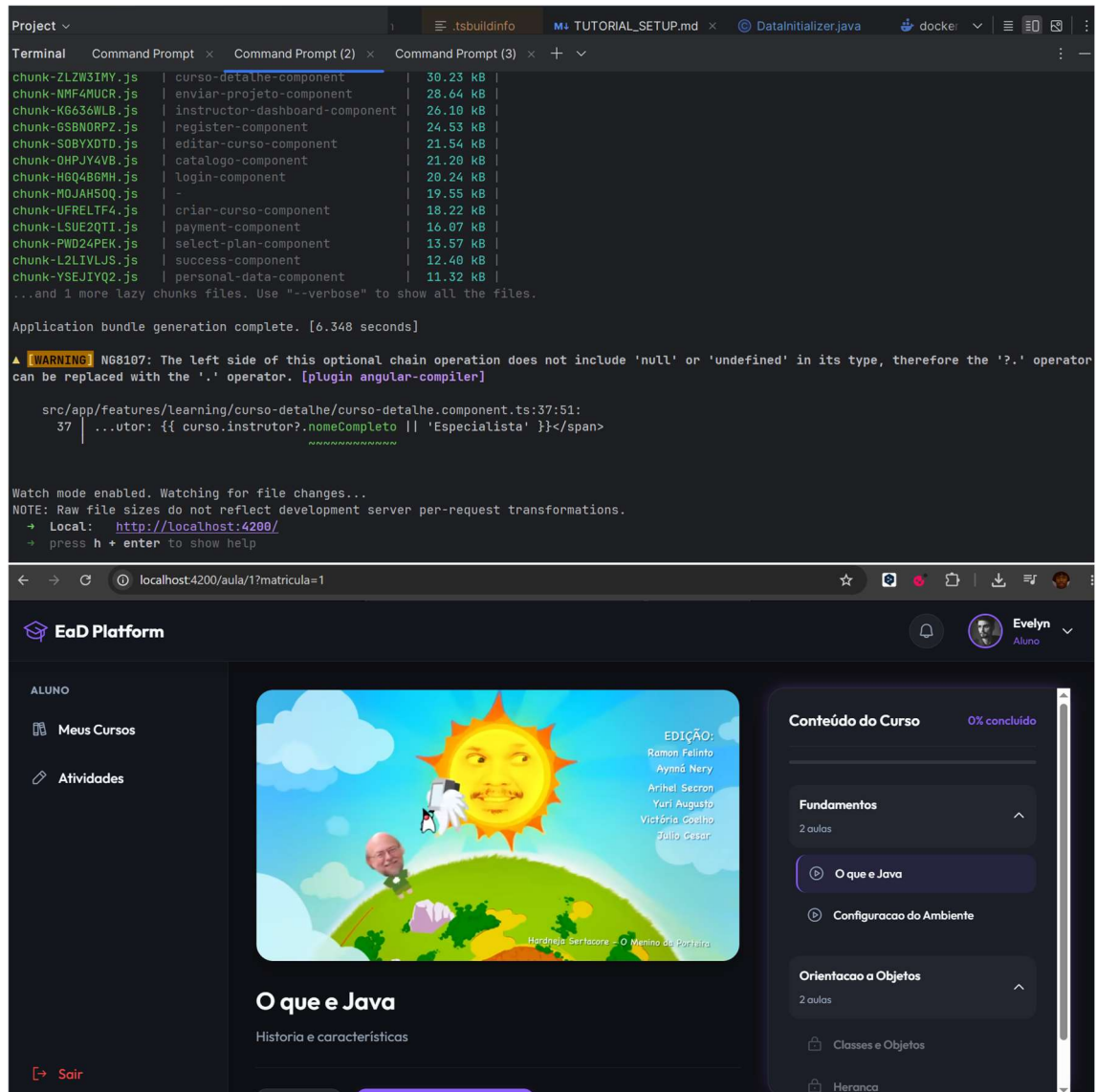




Tarefas Bônus (opcionais)

- 1) (0,5): integrar LLM Gemini na API.
- 2) (0,5): integrar com Front-End, trazendo a visão full-stack da aplicação.





3) (0,5): testar a aplicação com possíveis usuários – usuários deverão testar a aplicação: se matricular realizar um curso EAD e ganhar um benefício da plataforma. A aplicação deve estar hospedada em Docker containers.

Fluxo para o usuario testar:

1. Acessar <http://localhost:4200> (frontend) ou usar Swagger
2. Cadastrar/login com aluno@teste.com / 123456
3. Listar cursos em [GET /cursos](#)
4. Matricular-se em um curso via [POST /matriculas](#) com `{"cursoId": 1}`
5. Concluir modulos via [POST /matriculas/{id}/modulos/{ordem}/concluir](#)
6. Ao concluir todos os modulos, ganha 3 cursos extras e moedas (se Premium)
7. Após 12 cursos concluídos, faz upgrade para Premium automaticamente

Entregas – via CANVAS – Data de Entrega conforme Plano de Ensino

- 1) Link do projeto no Github, compartilhado com a professora. Colocar aqui, disponível para clone.
<https://github.com/Noxzxz/projeto-spring-data-usuario-simples>
- 2) Este documento devidamente preenchido conforme orientações acima, destacando as evidências correspondentes de cada questão.
- 3) O projeto deverá ser apresentado pela equipe para as professoras do projeto interdisciplinar. Apresentação de 20 minutos conforme template pptx. Todos devem apresentar a parte do projeto desenvolvida.
- 4) Tarefas Bônus (opcionais)
- 5) Cada integrante deve se apresentar com nome completo e RA e depois, explicar os endpoints realizados, a partir das camadas do projeto. Todos devem estar presentes na data da apresentação. A apresentação deve seguir os passos de 3 a 13 desta atividade.